Course: Cryptography and Network Security
Code: CSN15403
Branch: B.Tech -5th Semester

Lecture – 16 : Key Management

Faculty & Coordinator : Dr. J Sathish Kumar (JSK)

Department of Computer Science and Engineering

Motilal Nehru National Institute of Technology Allahabad,
Prayagraj-211004

SYMMETRIC-KEY DISTRIBUTION

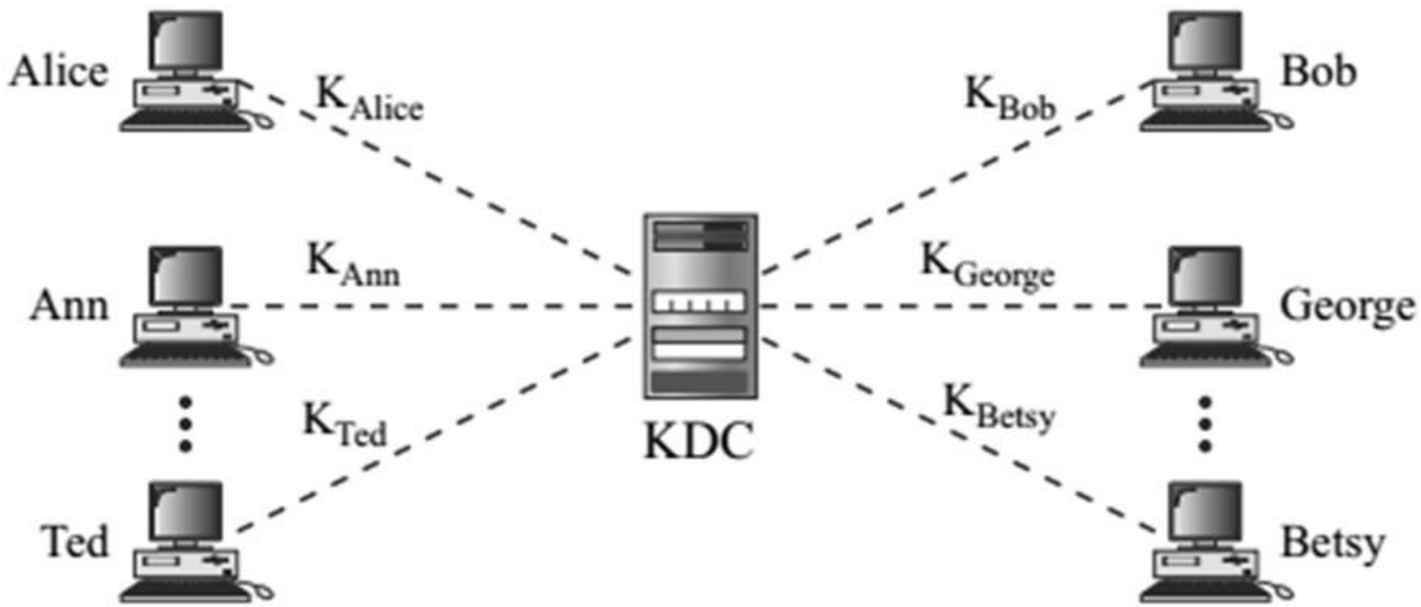
- If Alice needs to exchange confidential messages with N people, she needs N different keys.
- What if N people need to communicate with each other? A total of $N(N - 1)$ keys is needed if we require that Alice and Bob use two keys for bidirectional communication; only $N(N - 1)/2$ keys are needed if we allow a key to be used for both directions.
- This means that if one million people need to communicate with each other, each person has almost one million different keys; in total, almost one trillion keys are needed.
- This is normally referred to as the N^2 problem because the number of required keys for N entities is N^2.

SYMMETRIC-KEY DISTRIBUTION

- The number of keys is not the only problem; the distribution of keys is another.
- If Alice and Bob want to communicate, they need a way to exchange a secret key; if Alice wants to communicate with one million people, how can she exchange one million keys with one million people? Using the Internet is definitely not a secure method.
- It is obvious that we need an efficient way to maintain and distribute secret keys.

Key-Distribution Center: KDC

- A practical solution is the use of a trusted third party, referred to as a key-distribution center (KDC).
- To reduce the number of keys, each person establishes a shared secret key with the KDC



Key-Distribution Center: KDC

- A secret key is established between the KDC and each member. Alice has a secret key with the KDC, which we refer to as KAlice; Bob has a secret key with the KDC, which we refer to as KBob; and so on.
- Now the question is how Alice can send a confidential message to Bob. The process is as follows:
  - Alice sends a request to the KDC stating that she needs a session (temporary) secret key between herself and Bob.
  - The KDC informs Bob about Alice's request.
  - If Bob agrees, a session key is created between the two.

SYMMETRIC-KEY AGREEMENT

- Alice and Bob can create a session key between themselves without using a KDC.
- This method of session-key creation is referred to as the symmetric-key agreement.
- Although there are several ways to accomplish this, only two common methods, DiffieHellman and station-to-station, are discussed

SYMMETRIC-KEY AGREEMENT

- Alice and Bob can create a session key between themselves without using a KDC.
- This method of session-key creation is referred to as the symmetric-key agreement.
- Although there are several ways to accomplish this, only two common methods, DiffieHellman and station-to-station, are discussed

Diffie-Hellman Key Agreement

- In the Diffie-Hellman protocol two parties create a symmetric session key without the need of a KDC.
- Before establishing a symmetric key, the two parties need to choose two numbers p and g.
- The first number, p, is a large prime number on the order of 300 decimal digits (1024 bits).
- The second number, g, is a generator of order $p - 1$ in the group $\langle Zp^*, \times \rangle$.
- These two (group and generator) do not need to be confidential. They can be sent through the Internet; they can be public.

Diffie-Hellman Key Agreement

- The steps are as follows:

  1. Alice chooses a large random number x such that $0 \leq x \leq p - 1$ and calculates $R_1 = g^x \bmod p$.

  2. Bob chooses another large random number y such that $0 \leq y \leq p - 1$ and calculates $R_2 = g^y \bmod p$.

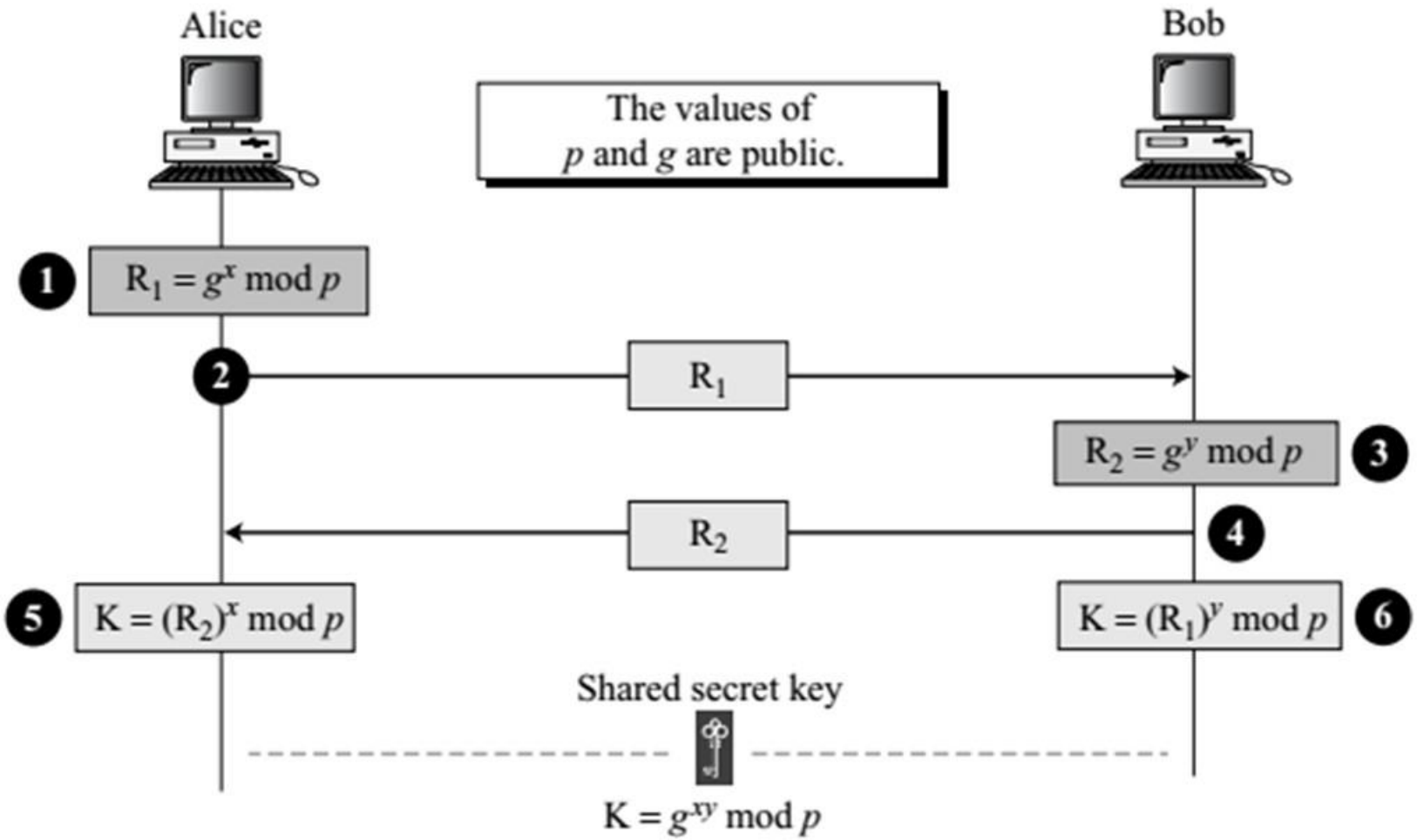
  3. Alice sends R_1 to Bob. Note that Alice does not send the value of x; she sends only R_1.

  4. Bob sends R_2 to Alice. Again, note that Bob does not send the value of y, he sends only R_2.

  5. Alice calculates $K = (R_2)^x \bmod p$.

  6. Bob also calculates $K = (R_1)^y \bmod p$

Diffie-Hellman Key Agreement



Diffie-Hellman Key Agreement

$$K = (g^x \bmod p)^y \bmod p = (g^y \bmod p)^x \bmod p = g^{xy} \bmod p$$

Bob has calculated $K = (R_1)^y \bmod p = (g^x \bmod p)^y \bmod p = g^{xy} \bmod p$.
Alice has calculated $K = (R_2)^x \bmod p = (g^y \bmod p)^x \bmod = g^{xy} \bmod p$.
Both have reached the same value without Bob knowing the value of x and without Alice knowing the value of y.

The symmetric (shared) key in the Diffie-Hellman method is $K = g^{xy} \bmod p$

Diffie-Hellman Key Agreement

Let us give a trivial example to make the procedure clear. Our example uses small numbers, but note that in a real situation, the numbers are very large. Assume that $g = 7$ and $p = 23$. The steps are as follows:

1. Alice chooses $x = 3$ and calculates $R_1 = 7^3 \bmod 23 = 21$.
2. Bob chooses $y = 6$ and calculates $R_2 = 7^6 \bmod 23 = 4$.
3. Alice sends the number 21 to Bob.
4. Bob sends the number 4 to Alice.
5. Alice calculates the symmetric key $K = 4^3 \bmod 23 = 18$.
6. Bob calculates the symmetric key $K = 21^6 \bmod 23 = 18$.

The value of K is the same for both Alice and Bob; $g^{xy} \bmod p = 7^{18} \bmod 35 = 18$.

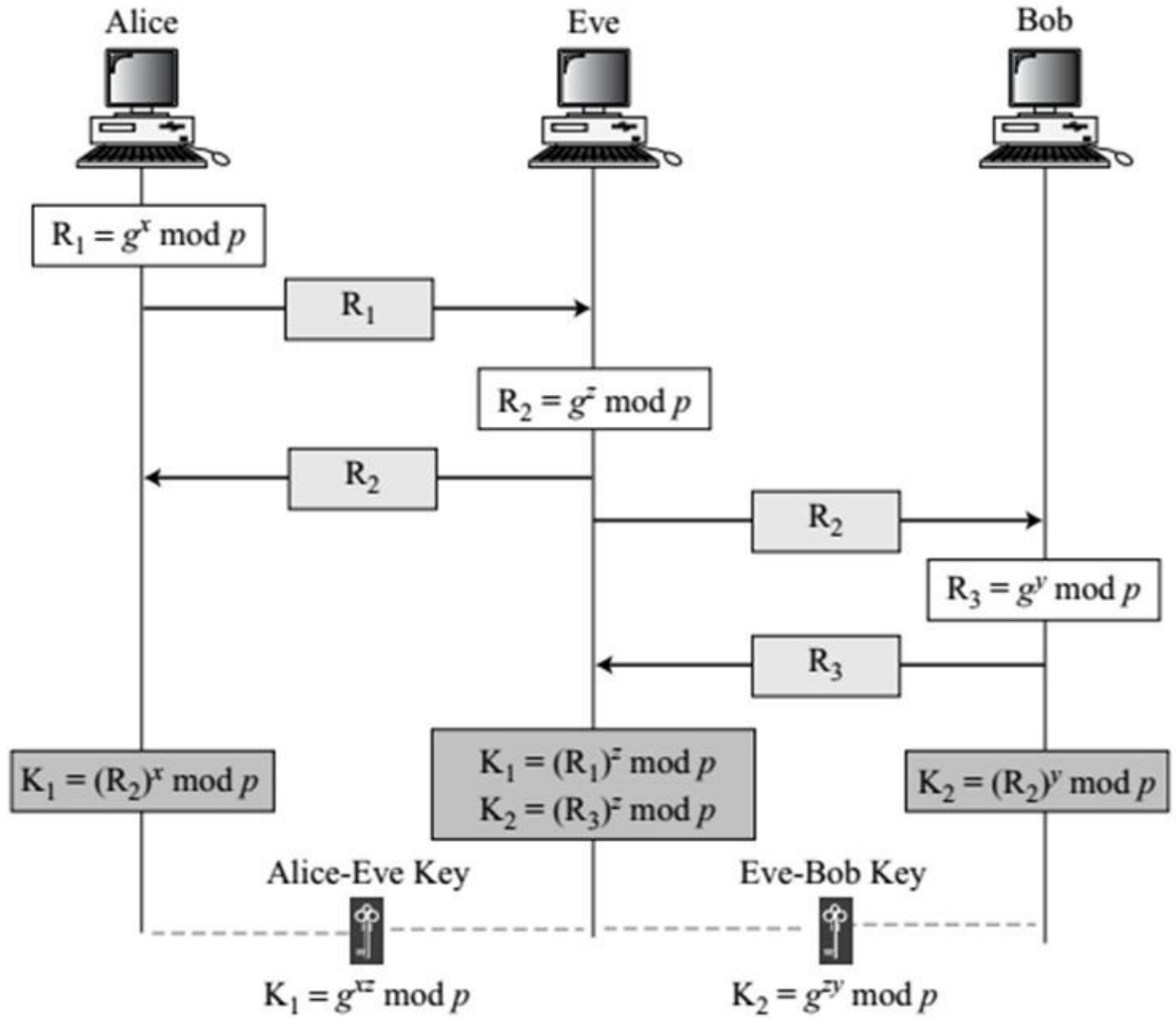
Security of Diffie-Hellman

- The Diffie-Hellman key exchange is susceptible to two attacks: the discrete logarithm attack and the man-in-the-middle attack
- Man-in-the-Middle Attack The protocol has another weakness. Eve does not have to find the value of x and y to attack the protocol.
- She can fool Alice and Bob by creating two keys: one between herself and Alice, and another between herself and Bob.

Security of Diffie-Hellman

- The following can happen:
- 1. Alice chooses x, calculates $R_1 = g^x \bmod p$, and sends R_1 to Bob.
- 2. Eve, the intruder, intercepts R_1. She chooses z, calculates $R_2 = g^z \bmod p$, and sends R_2 to both Alice and Bob.
- 3. Bob chooses y, calculates $R_3 = g^y \bmod p$, and sends R_3 to Alice. R_3 is intercepted by Eve and never reaches Alice.
- 4. Alice and Eve calculate $K_1 = g^{xz} \bmod p$, which becomes a shared key between Alice and Eve. Alice, however, thinks that it is a key shared between Bob and herself.
- 5. Eve and Bob calculate $K_2 = g^{zy} \bmod p$, which becomes a shared key between Eve and Bob. Bob, however, thinks that it is a key shared between Alice and himself.

Security of Diffie-Hellman

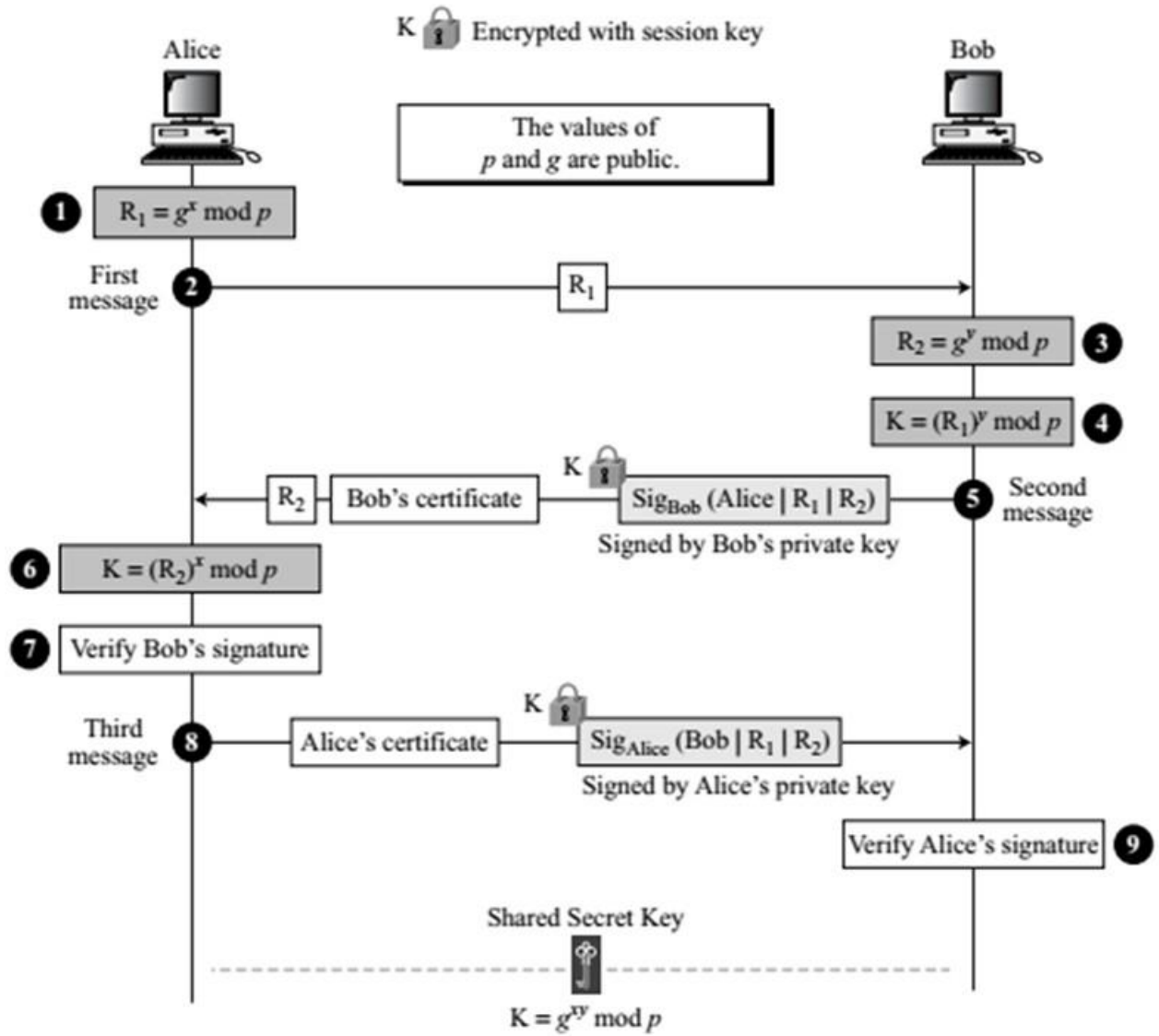


Security of Diffie-Hellman

- This situation is called a man-in-the-middle attack because Eve comes in between and intercepts R1, sent by Alice to Bob, and R3, sent by Bob to Alice.
- It is also known as a bucket brigade attack because it resembles a short line of volunteers passing a bucket of water from person to person.

Station-to-Station Key Agreement

- The station-to-station protocol
- is a method based on Diffie-
- Hellman.
- It uses digital signatures with
- public-key certificates to
- establish a session key between
- Alice and Bob



Station-to-Station Key Agreement

- The following shows the steps:
- After calculating R_1, Alice sends R_1 to Bob (steps 1 and 2 in Figure).
- After calculating R_2 and the session key, Bob concatenates Alice's ID, R_1, and R_2. He then signs the result with his private key. Bob now sends R_2, the signature, and his own public-key certificate to Alice. The signature is encrypted with the session key (steps 3, 4, and 5 in Figure).
- After calculating the session key, if Bob's signature is verified, Alice concatenates Bob's ID, R_1, and R_2. She then signs the result with her own private key and sends it to Bob. The signature is encrypted with the session key (steps 6, 7, and 8 in Figure).
- If Alice's signature is verified, Bob keeps the session key (step 9 in Figure)

Security of Station-to-Station Protocol

- The station-to-station protocol prevents man-in-the-middle attacks.
- After intercepting R_1, Eve cannot send her own R_2 to Alice and pretend it is coming from Bob because Eve cannot forge the private key of Bob to create the signature the signature cannot be verified with Bob's public key defined in the certificate.
- In the same way, Eve cannot forge Alice's private key to sign the third message sent by Alice.
- The certificates, are trusted because they are issued by trusted authorities.

Public-Key Infrastructures (PKI)

